

1. ¿Qué es Jest?

Jest es un entorno o framework de pruebas (*test runner*) de código abierto desarrollado y mantenido originalmente por Meta (Facebook). Es una solución integral (*all-in-one*) para JavaScript y TypeScript que proporciona:

- Un ejecutor de pruebas automático con soporte para ejecución paralela.
- Una librería de aserciones integrada (métodos como `expect(...)`, `toBe(...)`, `toEqual(...)`).
- Herramientas nativas para simulación (*mocking*) de módulos, funciones y temporizadores.
- Generador de métricas y cobertura de código (*code coverage*).
- Un entorno de simulación del navegador en Node.js llamado **jsdom**.

2. ¿Qué es React Testing Library (RTL)?

React Testing Library (RTL) es una biblioteca perteneciente a la familia *Testing Library*, creada por Kent C. Dodds. Proporciona utilidades para renderizar componentes de React e interactuar con ellos simulando el comportamiento real de un usuario final, en lugar de evaluar detalles internos de implementación técnica.

RTL no evalúa variables de estado interno ni nombres de métodos privados; en su lugar, busca elementos en el árbol del DOM simulado a través de roles accesibles (role), textos visibles o etiquetas de formulario (label).

3. Complementariedad: Jest vs. React Testing Library

Ambas herramientas trabajan en conjunto:

- **Jest** provee el andamiaje del test: define los bloques de prueba (describe, test/it), corre los archivos de testing y verifica las expectativas (expect).
- **React Testing Library** se encarga de montar el componente en el DOM virtual (render), consultar la interfaz visual (`screen.getByRole`, `screen.getByText`) y disparar interacciones reales (`fireEvent` o `userEvent`).

4. Características principales

- **Enfoque centrado en el usuario:** Se evalúa lo que el usuario ve y hace (clics, escritura de texto, lectura de mensajes), lo que hace que los tests sean resistentes a cambios y refactorizaciones internas de código.
- **Excelente soporte de accesibilidad (a11y):** Incentiva el uso de consultas semánticas (`getByRole`, `getByLabelText`, `getByPlaceholderText`), promoviendo interfaces accesibles.
- **Cero configuración inicial con herramientas modernas:** Proyectos configurados mediante Create React App o plantillas modernas con Vite/Jest vienen preconfigurados para reconocer extensiones como `.test.js` o `.test.jsx`.

- **Pruebas de instantáneas (*Snapshot Testing*):** Capacidad nativa de Jest para guardar una captura de la estructura renderizada y alertar ante cualquier cambio visual o estructural inesperado.
- **Simulación avanzada (*Mocking*):** Capacidad de aislar llamadas asíncronas a servicios de red (fetch, axios) sin necesidad de conectar a un backend real durante la ejecución de los tests.

5. ¿Para qué sirven?

Sirven para garantizar la calidad del software en el frontend mediante pruebas automatizadas (unitarias y de integración):

- Prevenir regresiones de software (detectar fallas de forma inmediata al refactorizar o añadir funciones).
- Documentar de forma viva el comportamiento esperado de cada componente.
- Verificar la validación de formularios y flujos interactivos de la interfaz de usuario.
- Facilitar procesos de integración continua (CI/CD) antes del despliegue en entornos productivos.

Ejemplo

Componente Contador.jsx

```
import React, { useState } from 'react';
function Contador() {
  const [cuenta, setCuenta] = useState(0);

  return (
    <div>
      <h2>Sección de Conteo</h2>
      <p data-testid="valor-cuenta">Valor: {cuenta}</p>
      <button onClick={() => setCuenta(cuenta +
1)}>Incrementar</button>
    </div>
  );
}
export default Contador;
```

Archivo de prueba Contador.test.jsx

```
import React from 'react';
import { render, screen, fireEvent } from '@testing-library/react';
import '@testing-library/jest-dom';
import Contador from '../Contador';

describe('Pruebas unitarias sobre el componente Contador', () => {
  test('Renderiza el componente con el valor inicial en 0', () => {
    //Renderizar el componente en el DOM simulado
    render(<Contador />);

    //Localizar elementos en pantalla
    const encabezado = screen.getByRole('heading', { name: /sección de conteo/i });
    const textoValor = screen.getByText(/valor: 0/i);

    //Aserciones con Jest
    expect(encabezado).toBeInTheDocument();
    expect(textoValor).toBeInTheDocument();
  });

  test('Incrementa el valor en 1 al pulsar el botón', () => {
    render(<Contador />);

    //Localizar el botón e interactuar simulando un clic del usuario
    const botonIncrementar = screen.getByRole('button', { name: /incrementar/i });
    fireEvent.click(botonIncrementar);

    //Verificar que el estado visual haya cambiado a 1
    const textoValorActualizado = screen.getByText(/valor: 1/i);
    expect(textoValorActualizado).toBeInTheDocument();
  });
});
```

Explicación del ejemplo:

1. describe y test: Funciones provistas por **Jest** para agrupar y definir cada caso de prueba de manera legible.
2. render(<Contador/>): Función de **React Testing Library** que monta el componente dentro del contenedor virtual provisto por jsdom.

3. `screen.getByRole(...)` y `screen.getByText(...)`: Métodos de consulta de **RTL** que buscan elementos en la pantalla tal como lo haría un usuario (por su etiqueta semántica o por el texto que lee).
4. `fireEvent.click(botonIncrementar)`: Simula la interacción real del usuario al hacer clic en el botón de incremento.
5. `expect(...).toBeInTheDocument()`: Aserción provista por Jest y la extensión `jest-dom` para verificar formalmente que el resultado esperado se encuentre montado y visible en el DOM. Si el texto o la actualización fallan, el test reportará un error indicando la discrepancia exacta.